DATABASE DOCUMENTATION FOR

LIBRARY MANAGEMENT SYSTEM

In Partial Fulfillment of the Requirements

for CC 105 – Information Management and CC106 – Application Development and Emerging Technologies

Prepared by:
GROUP 6 RENEVERSE

Members:
Mabilin, John Christian
Bechayda, Maryrose Angel
Tirao, Jessica
Briceno, Nica
Baesa, Roland Carl
Belga, John Eddrick

2026

INTRODUCTION

This paper presents the database documentation for the Library Management System developed by Group 6: Reneverse. The system uses a MySQL database named library_management to organize information about books, library branches, members, and borrowing transactions. The database also includes the users and book_requests tables, which support the login and book-request features implemented in the website/application.

The documentation discusses the database structure, relationships, data dictionary, normalization, and sample SQL queries. The Entity Relationship Diagram (ERD) and normalization discussion are limited to the four main tables: books, book_loans, members, and branches. The users and book_requests tables are discussed in the other relevant sections because they are used by the website/application developed by Group 6: Reneverse.

OBJECTIVE

- To store and manage book information, including ISBN, title, author, publisher, classification, location, and availability.
- To maintain library branch details and associate books, members, loans, and requests with relevant branches.
- To maintain member information and membership status.
- To record borrowing transactions, due dates, returns, loan status, renewals, and late-fine amounts.
- To support user registration, login, and role-based access through the users table.
- To record and manage book requests submitted by system users.
- To demonstrate CRUD operations, relational joins, filtering, aggregation, and reporting using SQL.

DATABASE OVERVIEW

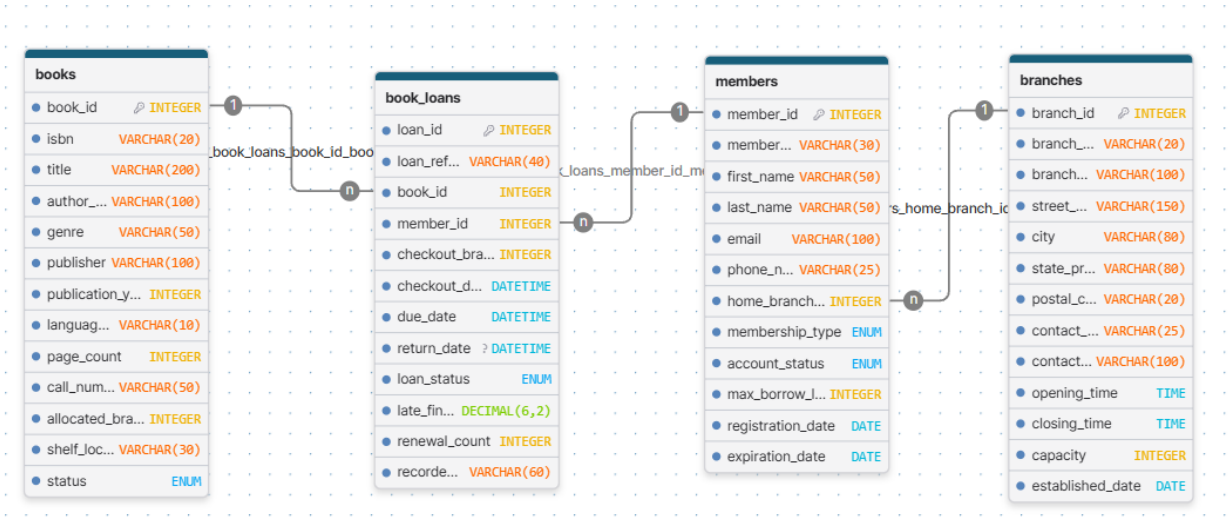
The library_management database consists of six tables: books, book_loans, branches, members, users, and book_requests. The books, book_loans, branches, and members tables represent the main library records. The users and book_requests tables support the website/application by storing account information and submitted borrowing requests.

	Purpose
books	Stores book information, classification, branch allocation, shelf location, and status.
book_loans	Stores borrowing transactions, due/return dates, loan status, and late fines.
branches	Stores library branch details and contact information.
members	Stores library member and membership information.
users	Stores website/application login accounts, usernames, password hashes, and roles.
book_requests	Stores users' book requests, selected pickup branch, request status, and review details.

Note: The users and book_requests tables are used in the website/application developed by the Group 6: Reneverse. The users table stores account details required for login and role-based access, while the book_requests table stores borrowing requests submitted through the application.

Entity Relationship Diagram (ERD)

The ERD presented in this section illustrates the relationships among books, book_loans, members, and branches. The users and book_requests tables are not included in the ERD.



This diagram shows how the library records are related. Books may be assigned to a branch, members may be associated with a home branch, and each book loan record connects a member with a borrowed book. The checkout branch is also recorded to identify the branch associated with the borrowing transaction.

DATABASE RELATIONSHIPS

The database uses primary keys and foreign keys to connect records across tables.

Related Tables	**Relationship Type**	**Primary / Foreign Key Link**	**Description**
Branches — Books	One-to-many (1:M)	branches.branch_id → books.allocated_branch_id	One branch can have many books assigned to it. Each book may be assigned to a branch.
Branches — Members	One-to-many (1:M)	branches.branch_id → members.home_branch_id	One branch can be the home branch of many members. A member's home branch is stored in home_branch_id.
Books — Book Loans	One-to-many (1:M)	books.book_id → book_loans.book_id	One book can appear in several loan records over time. Each loan record refers to a book.
Members — Book Loans	One-to-many (1:M)	members.member_id → book_loans.member_id	One member can have multiple borrowing records. Each loan record identifies the member who borrowed the book.
Branches — Book Loans	One-to-many (1:M)	branches.branch_id → book_loans.checkout_branch_id	One branch can be recorded for many checkout transactions. This field identifies the branch where the loan was processed.
Users — Book Requests (request owner)	One-to-many (1:M)	users.user_id → book_requests.user_id	One user account can submit multiple book requests. The user_id field identifies who submitted each request.

Books — Book Requests	One-to-many (1:M)	books.book_id → book_requests.book_id	One book can be referenced by multiple requests. Each request identifies the book being requested.
Branches — Book Requests	One-to-many (1:M)	branches.branch_id → book_requests.pickup_branch_id	One branch can be selected as the pickup location for many requests. The field stores the requested pickup branch.
Users — Book Requests (reviewer)	One-to-many (1:M)	users.user_id → book_requests.reviewed_by	One user account can review multiple requests. reviewed_by may be NULL until a request has been reviewed.

Note: The SQL schema does not define a direct foreign-key relationship between users and members. They serve different purposes in the current database: users is for application login, while members store library membership records if the users borrow a book (in application).

DATA DICTIONARY

The data dictionary lists the fields, data types, and key purposes based on the provided SQL schema. Enum fields are shown with their permitted values where available.

1. books Table

Field	Data type	Key / purpose
book_id	INT	Primary key; unique book record ID
isbn	VARCHAR(20)	Unique ISBN
title	VARCHAR(200)	Book title
author_name	VARCHAR(100)	Author name
genre	VARCHAR(50)	Book genre/category
publisher	VARCHAR(100)	Publisher name
publication_year	INT	Publication year
language_code	VARCHAR(10)	Language code; default ENG
page_count	INT	Number of pages
call_number	VARCHAR(50)	Library classification/call number

allocated_branch_id	INT	Foreign key to branches.branch_id
shelf_location	VARCHAR(30)	Physical shelf location
status	ENUM('AVAILABLE','CHECKED OUT','RESERVED','UNDER MAINTENANCE','LOST')	Available, Checked Out, Reserved, Under Maintenance, or Lost

2. book_loans Table

Field	**Data type**	**Key / purpose**
loan_id	INT	Primary key; unique loan ID
loan_reference	VARCHAR(40)	Unique loan reference
book_id	INT	Foreign key to books.book_id
member_id	INT	Foreign key to members.member_id
checkout_branch_id	INT	Foreign key to branches.branch_id
checkout_date	DATETIME	Date and time of checkout
due_date	DATETIME	Due date and time
return_date	DATETIME	Return date/time; may be NULL
loan_status	ENUM	Ongoing, Returned, Overdue, or Lost
late_fine_amount	DECIMAL(6,2)	Late fine amount
renewal_count	INT	Number of renewals
recorded_by_staff	VARCHAR(60)	Staff identifier/name recorded with the loan

3. branches Table

Field	**Data type**	**Key / purpose**
branch_id	INT	Primary key; branch ID
branch_code	VARCHAR(20)	Unique branch code
branch_name	VARCHAR(100)	Branch name
street_address	VARCHAR(150)	Street address
city	VARCHAR(80)	City

state_province	VARCHAR(80)	State/province
postal_code	VARCHAR(20)	Postal code
contact_phone	VARCHAR(25)	Branch phone
contact_email	VARCHAR(100)	Branch email
opening_time / closing_time	TIME	Operating hours
capacity	INT	Branch capacity
established_date	DATE	Date branch was established

4. members Table

Field	**Data type**	**Key / purpose**
member_id	INT	Primary key; member ID
membership_number	VARCHAR(30)	Unique membership number
first_name / last_name	VARCHAR(50)	Member name
email	VARCHAR(100)	Unique member email
phone_number	VARCHAR(25)	Member phone number
home_branch_id	INT	Foreign key to branches.branch_id
membership_type	ENUM	Student, Faculty, Regular, Senior, or Corporate
account_status	ENUM	Active, Suspended, Expired, or Pending
max_borrow_limit	INT	Maximum number of books allowed
registration_date / expiration_date	DATE	Membership dates

5. users Table

Supports login/sign-up and role-based access for the website/application.

Field	**Data type**	**Key / purpose**
user_id	INT UNSIGNED	Primary key; account ID
first_name / last_name	VARCHAR(50)	Account holder name
email	VARCHAR(100)	Unique email
username	VARCHAR(50)	Unique username

password	VARCHAR(255)	Password value stored by the application (hash)
role	ENUM('ADMIN','USER')	Admin or User; default User

6. book_requests Table

Stores borrowing requests created and reviewed through the website/application.

Field	**Data type**	**Key / purpose**
request_id	INT UNSIGNED	Primary key; primary key; unique request id
user_id	INT UNSIGNED	Foreign key to users.user_id
book_id	INT	Foreign key to books.book_id
pickup_branch_id	INT	Foreign key to branches.branch_id
request_note	VARCHAR(500)	Request note; defaults to empty string
status	ENUM('PENDING','APPROVED','REJECTED','CANCELLED')	Pending, Approved, Rejected, or Cancelled; defaults to Pending
request_date	DATETIME	Date/time the request was created
reviewed_by	INT UNSIGNED	Foreign key to users.user_id; nullable until

		reviewed
reviewed_at	DATETIME	Date/time the request was reviewed; may be NULL

DATABASE NORMALIZATION

Database normalization is the process of organizing data in a database to reduce unnecessary data duplication, maintain data consistency, and establish proper relationships between tables. It helps ensure that information is stored efficiently and that changes to the data can be managed without creating unnecessary inconsistencies.

The Library Management System database uses four main tables for this normalization analysis: books, branches, members, and book_loans. These tables use primary keys, unique keys, and foreign keys to identify records and establish relationships between related information.

FIRST NORMAL FORM (1NF)

First Normal Form (1NF) requires that each table has a primary key and that each field contains a single, atomic value rather than a repeating group or multiple values stored in one field.

In the Library Management System, the four main tables use individual columns for their attributes. For example, the books table stores the book title, author name, genre, publisher, publication year, and page count in separate fields. The members table also stores member information in separate columns, such as first name, last name, email, and phone number.

The book_loans table stores loan information, including the book ID, member ID, checkout date, due date, and loan status, in separate fields.

SECOND NORMAL FORM (2NF)

Second Normal Form (2NF) requires that a table is already in 1NF and that every non-key attribute depends on the whole candidate key, rather than only part of a composite key.

The main tables use individual primary keys:

- books — book_id
- branches — branch_id
- members — member_id
- book_loans — loan_id

These primary keys uniquely identify records in their respective tables. Because the shown tables use single-column primary keys, partial dependencies on part of a composite primary key do not apply to those primary keys.

THIRD NORMAL FORM (3NF)

Third Normal Form (3NF) requires that a table is already in 2NF and that non-key attributes do not depend transitively on a candidate key. In other words, non-key attributes should depend on the key, the whole key, and nothing but the key.

The Library Management System separates branch information into the branches table and references branch records through foreign keys in other tables.

For example:

- The books table uses allocated_branch_id to reference a branch.
- The members table uses home_branch_id to reference a branch.
- The book_loans table uses checkout_branch_id to reference a branch.

This structure allows branch information to be maintained in the branches table instead of repeating the complete branch details in every related book, member, or loan record.

Similarly, the book_loans table references the related book and member through book_id and member_id. This establishes relationships without requiring all book and member details to be duplicated in each loan record**.**

ANALYSIS OF THE MAIN DATABASE TABLES

Table	**Primary Key**	**Important Unique key or FKs**	**Purpose**
books	book_id	isbn (unique); allocated_branch_id (FK)	Stores book information and its allocated branch.
branches	branch_id	branch_code (unique)	Stores branch information, including address and contact details.
Members	member_id	email (unique); home_branch_id (FK)	Stores member information and the member's home branch.
Book_loans	loan_id	loan_reference	Stores loan transactions

		(unique); book_id, member_id, and checkout_branch_id (FKs)	and links them to books, members, and branches.

Note: *FK means foreign key. The unique constraints shown in the screenshots help prevent duplicate values in the corresponding columns.***EVIDENCE OF DATABASE STRUCTURE AND RELATIONSHIPS**

The following screenshots support the database structure, keys, and relationships implemented in the Library Management System.

1. **Books Table Structure**

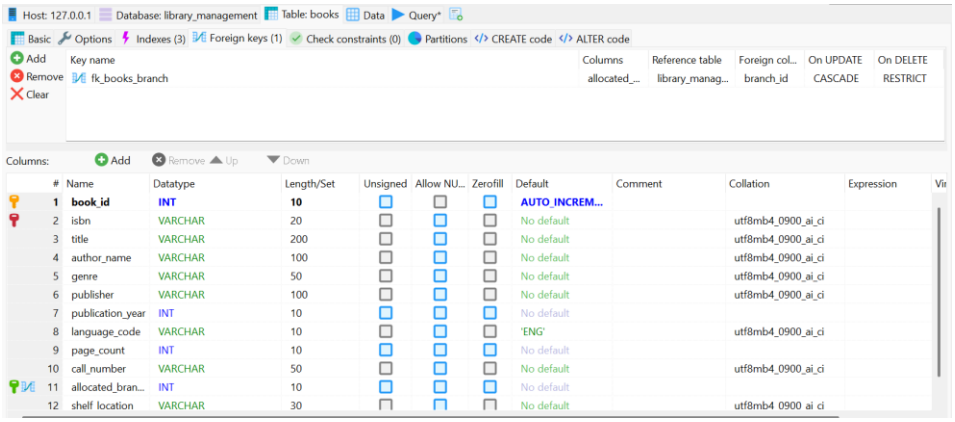


Figure 1. *Structure of the books table showing the book_id primary key, the unique isbn field, and the foreign key allocated_branch_id referencing the branches table.*

The screenshot shows that each book has a primary identifier and a unique ISBN constraint. It also shows the foreign key used to associate a book with a branch.

2. Branches Table Structure

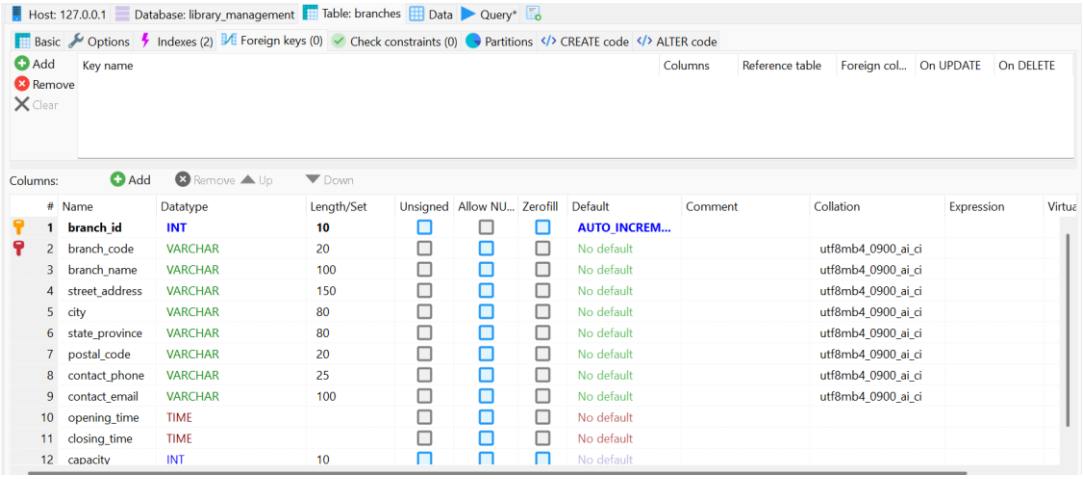


Figure 2. *Structure of the branches table showing branch_id as the primary key and branch_code as a unique field.*

3. Members Table Structures

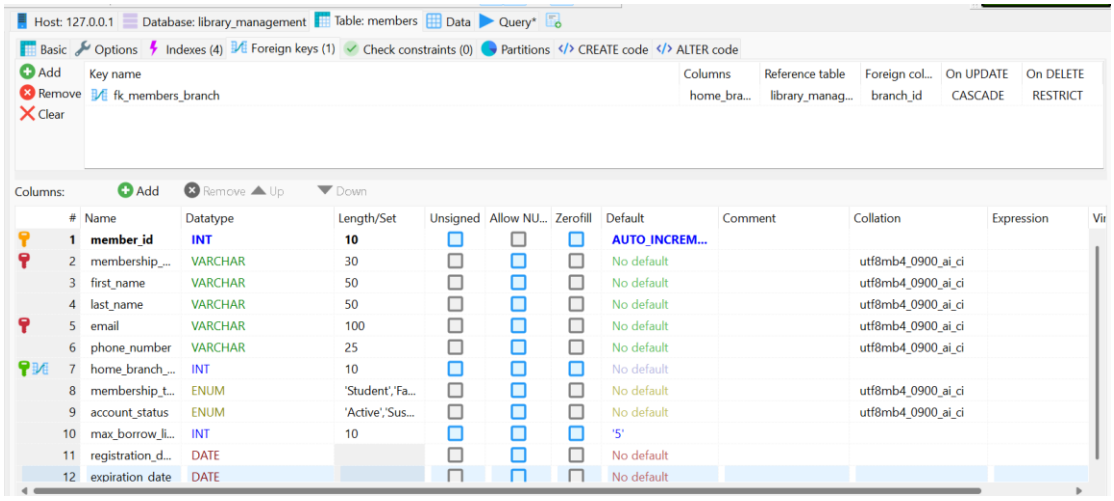


Figure 3. *Structure of the members table showing member_id as the primary key, email as a unique field, and home_branch_id as a foreign key referencing the branches table.*

The screenshot demonstrates how member information is stored separately from branch information. The foreign key establishes a relationship between a member and the member's home branch.

4. Book Loans Table Structure

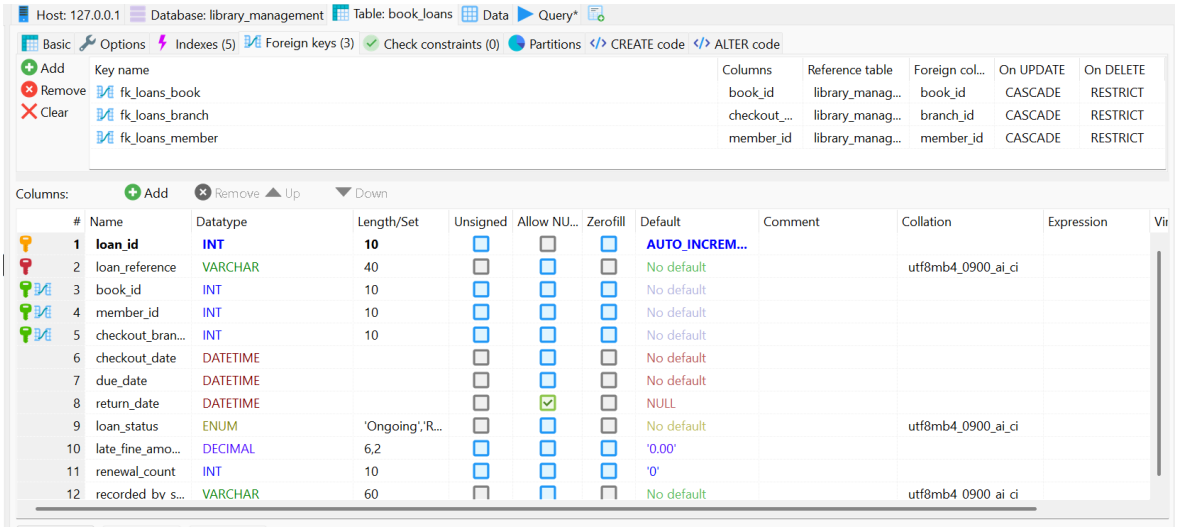


Figure 4. *Structure of the book_loans table showing loan_id as the primary key, loan_reference as a unique field, and foreign keys for the related book, member, and checkout branch.*

The screenshot shows that the book_loans table records borrowing transactions and connects them to the corresponding records in the books, members, and branches tables. These foreign key relationships help maintain referential integrity**.**

Summary of Database Relationships

The main relationships shown by the foreign keys are:

1. Branches and Books: A branch may be associated with multiple books through books.allocated_branch_id.
2. Branches and Members: A branch may be associated with multiple members through members.home_branch_id.
3. Books and Book Loans: A book may be referenced by loan records through book_loans.book_id.
4. Members and Book Loans: A member may be referenced by loan records through book_loans.member_id.
5. Branches and Book Loans: A branch may be referenced by loan records through book_loans.checkout_branch_id.

These relationships help the database connect related information while keeping the main entities in separate tables.

CONCLUSION

The Library Management System organizes library information through related tables for books, members, branches, borrowing transactions, and system users. Primary keys and foreign keys help maintain relationships between records, while SQL queries support searching, reporting, and monitoring of library activities. Normalization principles help reduce unnecessary duplication and improve data consistency. With a properly tested schema, sample data, and documented query results, the system can serve as a structured database solution for library operations.